Lab 07: Archived Web Pages and the Wayback Machine

INFO 4617/5617 · Week 7 · Chapter 7, Archived Web Pages and the Wayback Machine

Today we code together. Each step below says what we're building and why, and its code cell holds only the comments that frame it: we write the code live, in class. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

By the end you'll have read the Wayback Machine's addresses by hand, asked its two APIs which captures of a page exist, fetched the captures as they were saved, and measured how much a document changed from one capture to the next. The case is the chapter's: Facebook's Terms of Service, `facebook.com/terms`, from 2006 to 2024, with stops at Thefacebook in 2004 and CNN in 2000.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. Check that `import gensim` runs; if it doesn't, the chapter's "Document Similarity with Bag-of-Words" gives the one `conda install` line. The Wayback Machine limits how many requests one network address may send, and everyone in this room shares one, so run each cell once, when we do. If a cell stops with an error, the Archive may have turned us away: wait a minute and run it once more.

On Friday, run every cell above where we stopped (**Run → Run All Above Selected Cell**).

> **Instructor key.** Every cell is filled in, but most have not run. The key was run on 2026-10-21 from a cloud machine, following the textbook at commit `d9306b7`, and the Internet Archive refused that machine throughout: the Availability API answered every request with `429 Too Many Requests` (as HTML, so `response.json()` raised `JSONDecodeError`, as Step 3's Discuss box predicts), and `web.archive.org`, which serves the captures and the CDX Server, dropped every connection after about 11 seconds. Only the cells that need no capture ran: Step 0's imports, Step 1's parsing of `CAPTURE`, the definitions of `get_snapshot_stats()` and `get_wayback_content()`, and Step 10's stopword list, from GitHub. The other 21 code cells are tagged `not-run-live` and have no outputs, and Step 12's answers are not written. Before class, run the key with `labs/tools/run_key.py` on a laptop, then rewrite this note and Step 12's answers from its outputs. The CDX queries in Steps 5 and 7 end at past dates, so their counts should match a later rerun unless the Archive adds or withdraws captures for those dates; the CNN and news-site counts (Step 4) and the microsprints' pages depend on which capture the Availability API picks, which can change.

Step 0 — Set up (3 minutes)

Import what we'll use, and define the User-Agent every request will send, in the book's form. Every request today is a plain `requests.get()` with `headers=HEADERS`, as in week 1. The Internet Archive is a nonprofit that serves more than a trillion pages for free, and it is often slow: it answers `429 Too Many Requests` when one address sends too much. So every loop that sends requests pauses a second between them with `time.sleep(1)`, and we'll talk about what happens when the Archive says no.

```
# Import time, json, os, requests, pandas (as pd), matplotlib.pyplot (as plt), and BeautifulSoup
# Define HEADERS, the book's User-Agent: "WebDataScience/1.0 (your-email@colorado.edu)"
import time
import json
import os
import requests
import pandas as pd
import matplotlib.pyplot as plt
from bs4 import BeautifulSoup

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
```

💬 **Discuss**

- The Archive limits requests per network address, and this room shares one. What should thirty people running the same cell do differently from one person running it at home?
- Week 2 suggested one request a second as a default. Is that still polite when thirty of us send one at the same second?
- Week 3 called infrastructure that a community governs "ownership". Who pays for the requests we're about to send, and what do we owe them in return?

Step 1 — Read a capture's address (8 minutes)

Every capture has an address you can read and rewrite, as the chapter's "Moving Through Time" shows:

```
https://web.archive.org/web/20040212031928/http://www.thefacebook.com/
```

The 14 digits after `/web/` are the time of the capture in UTC, from year to second; everything after them is the original address. Parse the timestamp with `pd.to_datetime()` and a `format`. Then shorten the timestamp to a year: the Wayback Machine redirects you to the capture closest to it, so the address is itself a query. As in week 1, `response.history` keeps the trail of redirects.

```
# Define CAPTURE, the address above; split off its timestamp and original address (split on
 ↪  "/web/", then on the first "/")
# Print the timestamp as a UTC datetime (format="%Y%m%d%H%M%S", utc=True), and the original
 ↪  address
CAPTURE = "https://web.archive.org/web/20040212031928/http://www.thefacebook.com/"
stamp, original = CAPTURE.split("/web/")[1].split("/", 1)
print(pd.to_datetime(stamp, format="%Y%m%d%H%M%S", utc=True), original)
```

```
2004-02-12 03:19:28+00:00 http://www.thefacebook.com/
```

```
# Request "https://web.archive.org/web/2004/thefacebook.com" (a timestamp shortened to a year)
 ↪  with HEADERS
# Print each redirect in response.history (its status code and Location header), then the final
 ↪  status and address
response = requests.get("https://web.archive.org/web/2004/thefacebook.com", headers=HEADERS)
for step in response.history:
    print(step.status_code, step.headers["Location"])
print(response.status_code, response.url)
```

⏱️ Microsprint (5 minutes, alone or in pairs)

From the chapter's "Try It: Before They Were Famous" box.

Do: pick an address from the box (`google.com` in 1998, `x.com` in 1999) or a site you knew in middle school, and ask for it with only a year as the timestamp. Print where you landed and the page's `<title>`. **Share:** the address, the date of the capture you landed on, and its title.

```
# Microsprint: request "https://web.archive.org/web/<year>/<address>" for an address of your
 ↪  choice;
#   print response.url and the page's <title> (BeautifulSoup(response.text, "html.parser").title)
response = requests.get("https://web.archive.org/web/1999/x.com", headers=HEADERS)
print(response.url)
print(BeautifulSoup(response.text, "html.parser").title)
```

💬 Discuss

- `CAPTURE` was made at 03:19 UTC on February 12, 2004. What date and time was that at Harvard, where Thefacebook's users were?
- We asked for 2004 and landed on one capture. If you were studying a page in the week before an event, what could "closest" hide from you?

- Week 1's request to Wikipedia had an empty `response.history`. What is each redirect here telling us?

Step 2 — The archive's wrapper, and `id_` (6 minutes)

What came back in Step 1 isn't the page as it was captured. The Wayback Machine wraps it: it adds its toolbar and scripts, and rewrites the page's links and images to point back into the archive. Open `CAPTURE` in your browser, right-click, and **Inspect** it (week 5) to see the wrapper end at `<!-- END WAYBACK TOOLBAR INSERT -->`; the toolbar's **About this capture** shows when each image on the page was saved. That's convenient for reading and, as the chapter's "Inspecting Archived Pages" puts it, poisonous for measurement. Adding `id_` after the timestamp asks for the original capture, unmodified. Fetch both versions and count what each holds.

```
# Make RAW: CAPTURE with "id_" added after its timestamp
# For CAPTURE and then RAW: fetch and parse it, and print how many <script> tags and links (<a>
 ↪  with an href) it has,
#   and how many of the links point into the archive ("/web/" in the href); sleep 1 second. Then
 ↪  print RAW's first three links
RAW = CAPTURE.replace(stamp, stamp + "id_")
for version in [CAPTURE, RAW]:
    soup = BeautifulSoup(requests.get(version, headers=HEADERS).text, "html.parser")
    hrefs = [a["href"] for a in soup.find_all("a", href=True)]
    into_archive = [href for href in hrefs if "/web/" in href]
    print(f"{len(soup.find_all('script'))} scripts, {len(hrefs)} links, {len(into_archive)} into
     ↪  the archive: {version}")
    time.sleep(1)
print(hrefs[:3])
```

💬 Discuss

- Suppose you counted `<script>` tags on wrapped captures to study how JavaScript spread after 2004. What would you have found, and would anything have warned you?
- In week 5, the Inspector showed the page after scripts ran and View Source showed what the server sent. Which is `RAW` closer to, and which does your browser show you for a capture?
- Every link in `RAW` carries a `PHPSESSID`. What does that tell you about the crawler, and about what a logged-in student in 2004 would have seen instead?

Step 3 — The Availability API: one capture (5 minutes)

The Availability API answers one question in JSON: which capture of this address is closest to this date? Pass the address and the date as a `params` dictionary; `requests` encodes them, which matters because one of the values is itself a URL. Ask it about CNN's home page on August 15, 2000, take the capture it names from `["archived_snapshots"]["closest"]`, add `id_` as in Step 2, and count the capture's links, as the chapter's "The Availability API" does. The chapter first checks that the answer holds a capture; we assume it does.

```
# Request the Availability API ("https://archive.org/wayback/available") with HEADERS
#   and params url="www.cnn.com" and timestamp="20000815"; print its JSON
response = requests.get("https://archive.org/wayback/available",
                        params={"url": "www.cnn.com", "timestamp": "20000815"}, headers=HEADERS)
print(response.json())
```

```
# Take snapshot, the closest capture in the JSON, and make archived_url: its "url" with "id_"
 ↪  after its "timestamp"
# Fetch and parse archived_url; print the capture's timestamp and how many links (<a> tags) it
 ↪  has
snapshot = response.json()["archived_snapshots"]["closest"]
```

```
archived_url = snapshot["url"].replace(snapshot["timestamp"], snapshot["timestamp"] + "id_")
soup = BeautifulSoup(requests.get(archived_url, headers=HEADERS).text, "html.parser")
print(f"{snapshot['timestamp']}: {len(soup.find_all('a'))} links on CNN")
```

💬 Discuss

- We assumed the API found a capture. If `archived_snapshots` came back empty, as the chapter's "Common Issues to Debug" says it sometimes does when the Archive is busy, which line would break, and with what error?
- If the Archive answered `429 Too Many Requests` with a page of HTML, what would `response.json()` do? How do the chapter's `raise_for_status()` and `try`/`except`, and week 5's retrying session, guard against that?
- Week 4's JSON nested dictionaries inside dictionaries. Which keys did we walk through to reach the capture's address?

Step 4 — CNN across three eras, as a function (9 minutes)

To ask the same question about three dates, gather Step 3's lines into the chapter's `get_snapshot_stats(url, timestamp)`: ask the API, fetch the capture with `id_`, parse it, and return a dictionary of counts (links, text characters, images, and scripts). It's the chapter's function without its `try`/`except` and its `headers` parameter: it sends `HEADERS` itself. Run it for CNN in August 2000, 2012, and 2024, pausing a second between dates. The chapter's table of results is illustrative; ours will be real.

```
# Define get_snapshot_stats(url, timestamp): Step 3's lines, returning a dict with the "year"
 ↪  (timestamp[:4]) and the
#   "num_links", "text_length" (len(soup.get_text())), "num_images", and "num_scripts"
def get_snapshot_stats(url, timestamp):
    response = requests.get("https://archive.org/wayback/available",
                            params={"url": url, "timestamp": timestamp}, headers=HEADERS)
    snapshot = response.json()["archived_snapshots"]["closest"]
    archived_url = snapshot["url"].replace(snapshot["timestamp"], snapshot["timestamp"] + "id_")
    soup = BeautifulSoup(requests.get(archived_url, headers=HEADERS).text, "html.parser")
    return {"year": timestamp[:4], "num_links": len(soup.find_all("a")), "text_length":
     ↪  len(soup.get_text()),
            "num_images": len(soup.find_all("img")), "num_scripts": len(soup.find_all("script"))}
```

```
# Run get_snapshot_stats() for "www.cnn.com" at "20000815", "20120815", and "20240815", sleeping
 ↪  1 second between;
#   make the results a DataFrame, comparison, and print it
stats = []
for ts in ["20000815", "20120815", "20240815"]:
    stats.append(get_snapshot_stats("www.cnn.com", ts))
    time.sleep(1)
comparison = pd.DataFrame(stats)
print(comparison)
```

⏱️ Microsprint (5 minutes, alone or in pairs)

A page-statistics version of Additional Exercise 2, which compares two competing organizations.

Do: run `get_snapshot_stats()` for a competitor of CNN's (`www.nytimes.com`, `www.foxnews.com`, `www.usatoday.com`) in August 2000 and August 2024. **Share:** the site, and its numbers of links and scripts in each year.

```
# Microsprint: run get_snapshot_stats() for another news site at "20000815" and "20240815",
#   sleeping 1 second between; print the results as a DataFrame
rival = []
```

```
for ts in ["20000815", "20240815"]:
    rival.append(get_snapshot_stats("www.nytimes.com", ts))
    time.sleep(1)
print(pd.DataFrame(rival))
```

💬 Discuss

- `get_snapshot_stats()` reports the year we asked for, not the capture's own timestamp. How far from August 15 could the capture be, and how would you check?
- Which count grew most from 2000 to 2024? Week 6's "when 'static' isn't" warned about pages built by scripts: what can `requests` not see on the 2024 page?
- Each call sends two requests with no pause between them, then the loop sleeps a second. Is that within week 2's default of one request a second?

Step 5 — The CDX Server: every capture of a page (6 minutes)

The Availability API finds one capture; the CDX Server API lists all of them. Ask it about `facebook.com/terms` with the chapter's parameters: `output=json`, the fields (`fl`) we want, `filter=statuscode:200` for successful captures only, and `collapse=digest` to drop each capture whose content hash matches the one before it. We add one parameter the chapter leaves out, `to=20260930`, so that a rerun next year stops at the same date. The answer comes in one piece: the CDX Server pages its results only when you ask it to, so there's no token to follow. Its first row holds the column names.

```
# Request the CDX Server ("https://web.archive.org/cdx/search/cdx") with HEADERS and params:
 ↪  url="facebook.com/terms", output="json",
#   fl="timestamp,original,statuscode,length", filter="statuscode:200", collapse="digest", and
 ↪  to="20260930"
# Read the JSON into data; print how many rows it has, and its first three
cdx_url = "https://web.archive.org/cdx/search/cdx"
params = {"url": "facebook.com/terms", "output": "json", "fl":
 ↪  "timestamp,original,statuscode,length",
          "filter": "statuscode:200", "collapse": "digest", "to": "20260930"}
data = requests.get(cdx_url, params=params, headers=HEADERS).json()
print(len(data), data[:3])
```

```
# Make df, a DataFrame of rows 1 onward with row 0 as the columns; convert timestamp (format
 ↪  "%Y%m%d%H%M%S")
#   and length (pd.to_numeric, errors="coerce"); print the number of captures and the date range
df = pd.DataFrame(data[1:], columns=data[0])
df["timestamp"] = pd.to_datetime(df["timestamp"], format="%Y%m%d%H%M%S")
df["length"] = pd.to_numeric(df["length"], errors="coerce")
print(f"Total snapshots: {len(df)}")
print(f"Date range: {df['timestamp'].min()} to {df['timestamp'].max()}")
```

💬 Discuss

- How many requests did that take? How many would the Availability API need to tell us the same thing?
- If the CDX Server answered `429`, or dropped the connection as it sometimes does, which line of the first cell would fail, and would its error tell you why?
- Week 4's JSON often came as a list of dictionaries. What shape is this JSON, and why did we need `data[0]`?

Step 6 — Plot the archival record (6 minutes)

Two questions for the record: how often was a new version captured, and how big was each one? Set the timestamp as the index and `resample()` by quarter (`"QS"`), counting captures and averaging

`length`, as the chapter's "Filtering and Analyzing CDX Data" does. Plot both, one above the other, on a shared time axis. The CDX Server's `length` is the size of the archived record, compressed, not the size of the page in your browser: good for spotting jumps, not for saying how big the page was.

```
# Make df_ok, df indexed by timestamp; resample it by quarter ("QS") into quarterly, with
#   captures=("length", "size") and mean_size=("length", "mean"); print the three busiest
 ↪  quarters
df_ok = df.set_index("timestamp")
quarterly = df_ok.resample("QS").agg(captures=("length", "size"), mean_size=("length", "mean"))
print(quarterly.nlargest(3, "captures"))
```

```
# Plot captures and mean_size one above the other (plt.subplots(2, 1, sharex=True)), each with a
 ↪  title and a y label
fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
quarterly["captures"].plot(ax=axes[0], title="facebook.com/terms: distinct captures (status 200,
 ↪  collapsed on digest)")
axes[0].set_ylabel("Captures per quarter")
quarterly["mean_size"].plot(ax=axes[1], title="facebook.com/terms: mean size of the archived
 ↪  record")
axes[1].set_ylabel("Bytes, compressed")
axes[1].set_xlabel("Quarter")
plt.show()
```

💬 Discuss

- With `collapse=digest`, every row is a change in bytes. Is the busiest quarter the quarter Facebook changed its terms most, or the quarter the Archive paid most attention?
- Pick a sudden jump in size. What would you open to tell a redesign from a move behind a login?
- In week 1 we plotted an article's daily views. Whose behavior does this plot measure: Facebook's, its readers', or the Archive's?

Step 7 — What the calendar's colors mean (9 minutes)

The Wayback calendar colors each day by the HTTP status code (week 5) the crawler received: blue for 2xx, green for 3xx, orange for 4xx, red for 5xx. The CDX Server reports the same codes in `statuscode`, so drop the filter and look at what Step 5 left out. Ask for every capture of `facebook.com` from 2005, the year the address changed hands, and cross-tabulate month by status code with `pd.crosstab()`. The chapter's "What the Colors Mean" says what to expect from April to August.

```
# Request the CDX Server with params url="facebook.com", output="json",
 ↪  fl="timestamp,statuscode",
#   from="2005", and to="2005", with no filter and no collapse
# Make fb2005, a DataFrame, add a month column (the timestamp's first 6 characters), and print
 ↪  pd.crosstab() of month by statuscode
params = {"url": "facebook.com", "output": "json", "fl": "timestamp,statuscode", "from": "2005",
 ↪  "to": "2005"}
data = requests.get(cdx_url, params=params, headers=HEADERS).json()
fb2005 = pd.DataFrame(data[1:], columns=data[0])
fb2005["month"] = fb2005["timestamp"].str[:6]
print(pd.crosstab(fb2005["month"], fb2005["statuscode"]))
```

⏱️ Microsprint (5 minutes, alone or in pairs)

A status-code version of Additional Exercise 4, on a page the chapter's EDGI study watched.

Do: drop the filter for `epa.gov/climatechange`, the EPA's climate change page, from 2016 to 2018, and cross-tabulate year by status code. **Share:** the year its captures stop being 200s, and the status codes that replaced them.

```
# Microsprint: the same unfiltered query for url="epa.gov/climatechange", from "2016" to "2018";
#   cross-tabulate the year (the timestamp's first 4 characters) by statuscode
params = {"url": "epa.gov/climatechange", "output": "json", "fl": "timestamp,statuscode", "from":
 ↪  "2016", "to": "2018"}
data = requests.get(cdx_url, params=params, headers=HEADERS).json()
epa = pd.DataFrame(data[1:], columns=data[0])
print(pd.crosstab(epa["timestamp"].str[:4], epa["statuscode"]))
```

💬 Discuss

- The Archive recorded each refusal accurately. For which research question are the 403 rows noise, and for which are they the finding?
- A soft 404 is a ”Page not found” page sent with a 200. Would Step 5's filter keep it or drop it, and how would you catch one?
- In week 3's dead-endpoint forensics, the way an endpoint failed was the finding. What does the status column tell us here that the pages themselves can't?

Step 8 — One version's text, as a function (8 minutes)

To measure what changed, we need the words, not the sizes. The chapter's `get_wayback_content(url, date)` is Step 4's function with three differences: it takes a date and formats it with `strftime("%Y%m%d")`, it keeps the capture's own timestamp, and it returns the visible text, from BeautifulSoup's `get_text(separator=" ", strip=True)`, with its word count. As in Step 4, we leave out the chapter's `try`/`except` and `headers` parameter. Try it on one date, the start of 2010.

```
# Define fb_tos_url = "facebook.com/terms", and get_wayback_content(url, date): like
 ↪  get_snapshot_stats(), but with the
#   timestamp date.strftime("%Y%m%d"), and returning the capture's "timestamp", its id_ "url",
 ↪  the "word_count",
#   and the text as "content" (soup.get_text(separator=" ", strip=True))
fb_tos_url = "facebook.com/terms"

def get_wayback_content(url, date):
    response = requests.get("https://archive.org/wayback/available",
                            params={"url": url, "timestamp": date.strftime("%Y%m%d")},
 ↪  headers=HEADERS)
    snapshot = response.json()["archived_snapshots"]["closest"]
    archived_url = snapshot["url"].replace(snapshot["timestamp"], snapshot["timestamp"] + "id_")
    soup = BeautifulSoup(requests.get(archived_url, headers=HEADERS).text, "html.parser")
    text = soup.get_text(separator=" ", strip=True)
    return {"timestamp": snapshot["timestamp"], "url": archived_url, "word_count":
     ↪  len(text.split()), "content": text}
```

```
# Call get_wayback_content() for fb_tos_url at pd.Timestamp("2010-01-01");
#   print the capture's timestamp, its word count, and its first 300 characters
version = get_wayback_content(fb_tos_url, pd.Timestamp("2010-01-01"))
print(version["timestamp"], version["word_count"], "words")
print(version["content"][:300])
```

⏱️ Microsprint (5 minutes, alone or in pairs)

A short version of Additional Exercise 1, which tracks a policy document of your choice.

Do: pick another policy page (another platform's terms, such as `twitter.com/tos`, or a university's privacy policy) and run `get_wayback_content()` on it at the start of 2012 and of 2022. **Share:** the page, the dates of the two captures you got, and their word counts.

```
# Microsprint: run get_wayback_content() for another policy page at 2012-01-01 and 2022-01-01,
#   sleeping 1 second between; print each capture's timestamp and word count
for day in ["2012-01-01", "2022-01-01"]:
    other = get_wayback_content("twitter.com/tos", pd.Timestamp(day))
    print(other["timestamp"], other["word_count"], "words")
    time.sleep(1)
```

💬 Discuss

- If a date had no capture, `archived_snapshots` would be empty. What would our `get_wayback_content()` do then, and what does the chapter's version return instead?
- `get_text()` keeps every word on the page, menus and footers included. What will that do to the comparisons in Steps 10 and 11?
- Week 2 called terms of service "quasi-law". Who besides researchers would want every past version of one, and why?

Step 9 — Collect the history, with a cache (6 minutes)

Now the loop. The chapter asks for every quarter from 2005 to 2024: 77 dates and more than 150 requests, which from this room would take longer than the lab and strain the Archive. We take the start of every other year from 2006 to 2024, ten dates. As the chapter's "Caching Raw Captures" does, save each result as a JSON file in `wayback_cache/`, named by its date, with `json.dump()`, and skip any date whose file already exists, so that rerunning the loop on Friday asks the Archive for nothing. The chapter wraps the cache in a second function, `get_wayback_content_cached()`; we write its lines straight into the loop. Then load the files back with `json.load()`, as the chapter's "Serializing Data" does.

```
# Make the folder "wayback_cache"; make dates, pd.date_range("2006-01-01", "2024-01-01",
↪  freq="2YS")
# For each date: if "wayback_cache/YYYYMMDD.json" doesn't exist yet, call get_wayback_content()
↪  for fb_tos_url,
#   json.dump() the result into that file, and sleep 1 second
os.makedirs("wayback_cache", exist_ok=True)
dates = pd.date_range("2006-01-01", "2024-01-01", freq="2YS")
for date in dates:
    cache_path = f"wayback_cache/{date.strftime('%Y%m%d')}.json"
    if not os.path.exists(cache_path):
        result = get_wayback_content(fb_tos_url, date)
        with open(cache_path, "w") as f:
            json.dump(result, f)
        time.sleep(1)
```

```
# Load each date's file back into a list, results, with json.load(); print each version's
↪  timestamp and word count
results = []
for date in dates:
    with open(f"wayback_cache/{date.strftime('%Y%m%d')}.json") as f:
        results.append(json.load(f))
for r in results:
    print(r["timestamp"], r["word_count"], "words")
```

💬 Discuss

- The cache names each file by date alone. What happens if you run the loop for `twitter.com/tos` on the same dates?
- Did two of our dates land on the same capture? What would that do to Step 11's change timeline?

- The loop fetches a result before it opens the file. If it opened the file first and the request failed, what would be left in `wayback_cache/`, and what would the next run do?

Step 10 — Bag of words: the first version and the last (6 minutes)

A bag of words represents a document by how often each word appears in it, ignoring their order. First fetch the book's list of 198 stopwords, words like the and of that are too common to tell versions apart, from `https://raw.githubusercontent.com/cuinfoscience/Web-Data-Science-Book/main/data/stopwords-en.txt`. gensim's `simple_preprocess()` splits a text into lowercase tokens, and we drop the stopwords. A gensim `Dictionary` numbers the words, `doc2bow()` turns each document into a bag, and `SparseMatrixSimilarity` gives their cosine similarity: 1 for the same words in the same proportions, 0 for no words in common.

```
# Import simple_preprocess from gensim.utils, and corpora and similarities from gensim
# Fetch the stopword list (the address above) with HEADERS, make stop_words a set of its words,
 ↪  and print how many there are
from gensim.utils import simple_preprocess
from gensim import corpora, similarities

STOPWORDS_URL =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/Web-Data-Science-Book/main/data/stopwords-en.txt"
stop_words = set(requests.get(STOPWORDS_URL, headers=HEADERS).text.split())
print(f"{len(stop_words)} stopwords, such as {sorted(stop_words)[:5]}")
```

```
198 stopwords, such as ['a', 'about', 'above', 'after', 'again']
```

```
# Make doc_v1 and doc_v2: the first and last versions' content, through simple_preprocess(),
 ↪  without the stopwords
# Build a Dictionary and a doc2bow corpus from the two, index them with SparseMatrixSimilarity,
 ↪  and print their similarity
doc_v1 = [token for token in simple_preprocess(results[0]["content"]) if token not in stop_words]
doc_v2 = [token for token in simple_preprocess(results[-1]["content"]) if token not in
 ↪  stop_words]
dictionary = corpora.Dictionary([doc_v1, doc_v2])
corpus = [dictionary.doc2bow(doc) for doc in [doc_v1, doc_v2]]
index = similarities.SparseMatrixSimilarity(corpus, num_features=len(dictionary))
print(f"{len(doc_v1)} and {len(doc_v2)} tokens; similarity {index[corpus[0]][1]:.3f}")
```

💬 Discuss

- not, no, and nor are on the stopword list. Think of a sentence in a terms of service whose meaning flips when not is added. Would our score notice?
- Menus, footers, and cookie notices appear in both versions. Do they push the score toward 1 or toward 0?
- Week 6's parsers kept one table or one infobox; this one keeps every word on the page. When is each the right choice?

Step 11 — A change timeline (9 minutes)

Now every version. Gather Step 10's token line into the chapter's `preprocess_document(text)`, and run it on all ten. Compare each version with the one before it, and call one minus their similarity the change score: 0 for no change, 1 for nothing in common. Build one `Dictionary` and one index for all the versions, as the chapter's "Visualizing Document Change Over Time" does. The chapter plots the scores against the transition's number; we plot each against the date of the later capture, so a spike can be matched with an event.

```
# Define preprocess_document(text): Step 10's line, simple_preprocess() without the stopwords
# Preprocess every version in results into processed_docs; build one Dictionary, corpus, and
 ↪  SparseMatrixSimilarity index
```

```
def preprocess_document(text):
    return [token for token in simple_preprocess(text) if token not in stop_words]

processed_docs = [preprocess_document(r["content"]) for r in results]
dictionary = corpora.Dictionary(processed_docs)
corpus = [dictionary.doc2bow(doc) for doc in processed_docs]
index = similarities.SparseMatrixSimilarity(corpus, num_features=len(dictionary))
```

```
# For each version after the first, compute its change score: 1 minus its similarity to the
 ↪  version before (index[corpus[i-1]][i])
# Plot the scores against the later capture's date, with markers, a title, and labeled axes;
 ↪  print them
change_scores = [1 - index[corpus[i-1]][i] for i in range(1, len(corpus))]
when = pd.to_datetime([r["timestamp"] for r in results[1:]], format="%Y%m%d%H%M%S")
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(when, change_scores, marker="o")
ax.set(xlabel="Date of the later capture", ylabel="Change score (0 = identical)",
 ↪  title="facebook.com/terms: change from the version before")
plt.show()
print(pd.Series(change_scores, index=when.date).round(3))
```

⏱️ Microsprint (6 minutes, alone or in pairs)

A short version of Additional Exercise 3, which compares every version with every other.

Do: compute the similarity of every version with every other (`index[corpus]` is the whole matrix) and draw it as a heatmap with `imshow()`, both axes labeled with the capture years. **Share:** the pair of years whose versions differ most, and any run of years where the page stayed the same.

```
# Microsprint: matrix = index[corpus]; draw it with ax.imshow(), label both axes with the capture
 ↪  years,
#   and add a colorbar and a title
matrix = index[corpus]
years = [r["timestamp"][:4] for r in results]
fig, ax = plt.subplots(figsize=(6, 5))
image = ax.imshow(matrix)
ax.set_xticks(range(len(years)), years, rotation=90)
ax.set_yticks(range(len(years)), years)
ax.set(xlabel="Year of capture", ylabel="Year of capture", title="facebook.com/terms: similarity
 ↪  of every pair of versions")
fig.colorbar(image, label="Cosine similarity")
plt.show()
```

💬 Discuss

- Pick the biggest spike. What happened at Facebook around then, and how would you check it with archived sources rather than from memory?
- Captures two years apart can't see a change that came and went between them. What would the chapter's quarterly dates add, and what would they cost the Archive?
- The chapter's graduate exercise warns that observed change is confounded with attention. Where could that confound enter this plot?

Step 12 — What did we learn? (4 minutes)

In a few sentences, as comments or in a Markdown cell:

- How many distinct versions of `facebook.com/terms` did the CDX Server list in Step 5, and how

many different captures did our ten dates reach in Step 9? What does the gap mean for Step 11's timeline?
- In Step 7, what share of `facebook.com`'s captures in 2005 weren't 200s? What would a study that kept only the 200s have missed?
- Which transition changed most in Step 11? Does it show up as a jump in Step 6's sizes?
- What would you write in a methods section about how the Wayback Machine shaped these results?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, query the CDX Server for your page's history, and handle a 429: Step 5's query; the waiting and retrying is yours to write (Steps 3 and 5's Discuss questions).
- Step 2, build the capture DataFrame and print its date range and size: Step 5.
- Step 3, plot captures per year and page size over time: Step 6's `resample()` and two-panel plot.
- Step 4, fetch the earliest and latest captures with `id_` and cache them: Step 2's `id_`, and Step 9's cache.
- Step 5, extract the text of both captures and measure their similarity: Step 8's `get_text()`, and Step 10.
- Step 6, interpret: Steps 6, 7, and 11, and the Discuss questions about gaps and capture bias.

```
# Your answers here, as comments
# ANSWERS 1-3 TO BE WRITTEN AFTER A LIVE RUN. The Wayback Machine refused this key's run
#   (see the key note), so Steps 5-11 have no outputs to answer from. Each answer needs:
# 1. [Step 5: rows in df] distinct versions; [Step 9: distinct timestamps in results]
#   different captures reached by our ten dates.
# 2. [Step 7: non-200 captures / all captures] of facebook.com's 2005 captures weren't 200s
#   (the chapter's calendar shows 403s from April 10 to early August).
# 3. [Step 11: the date of the highest change score], against Step 6's mean_size then.
# 4. A methods section would say that the Archive's attention, not the page, set which versions
#   we saw: the crawl schedule, collapse=digest, and the capture closest to each date. It would
#   also say who could collect at all: on 2026-10-21 the Archive refused every request from this
#   key's cloud machine, with 429s and dropped connections, so where a study runs from is part
#   of its method.
```

Friday: from the lab to the textbook (15 minutes)

What did the lab turn up that the chapter should know? Some places to look:

- Step 4's table, next to the chapter's illustrative one in ”Comparing Multiple Snapshots”. Should the chapter print real output, with its date?
- Step 9's cache, which names files by date alone (the Discuss question). Is that a bug in the chapter's code, and what would you name the files instead?
- The chapter wraps every request in `try`/`except` with `raise_for_status()`; we left them out. After today, which guard would you keep, and should the chapter say why it's there?
- The chapter's collection loop, which calls `get_wayback_content()` without `headers`, so its requests go out with no User-Agent of ours.
- Step 6's `length`, which the chapter calls ”page size”. Is ”compressed record size” clearer, or just more words?
- A cell that failed today, and what the chapter's ”Common Issues to Debug” should say about it.
- A microsprint that would make a good exercise.

`handouts/common/revision-framework.md` says how to propose a revision.